

MNIST to Spike Train Encoding Guide

Sheng Lu

May 3, 2026

1 Overview

This document describes the preprocessing pipeline that converts the MNIST dataset into:

- 8×8 downsampled images
- Rate-based Poisson spike trains
- HSPICE-compatible PWL input signals

This pipeline bridges **image data** and **hardware SNN simulation**.

2 Processing Pipeline

1. Download MNIST dataset
2. Load images and labels
3. Resize images from 28×28 to 8×8
4. Convert pixel intensity to spike trains
5. Export spike trains to SPICE PWL format

3 Key Parameters

3.1 Time Configuration

```
duration_us = 100;  
dt_us = 1;  
T = duration_us / dt_us;
```

Parameter	Description
duration_us	Total duration per sample
dt_us	Time resolution
T	Number of time steps

3.2 Spike Encoding Parameters

```
max_spikes = 40;
```

- Pixel value = 1 produces up to 40 spikes
- Linear mapping between intensity and firing rate

4 Image Preprocessing

4.1 Resize to 8×8

```
images8(:,:,i) = imresize(images(:,:,i), [8 8], 'bilinear');  
images8 = images8 / 255;
```

Purpose:

- Reduce dimensionality ($784 \rightarrow 64$)
- Match hardware input size

5 Rate-Based Poisson Encoding

5.1 Principle

Each pixel is treated as a Poisson spike generator:

$$\lambda_i = I_i \cdot \text{max_spikes}$$

$$p_i = \frac{\lambda_i}{T}$$

- I_i : normalized pixel intensity
- λ_i : expected spike count
- p_i : probability of spike at each timestep

5.2 Implementation

```
lambda = img_vec * max_spikes;  
p_spike = lambda / T;  
  
for t = 1:T  
    spikes(:, t, n) = rand(num_pixels, 1) < p_spike;  
end
```

Properties:

- Stochastic encoding
- Preserves intensity in expectation
- Suitable for STDP learning

6 Alternative: Uniform Encoding

- Deterministic spike distribution
- Evenly spaced spikes

```
spike_times = round(linspace(1, T, spike_count(i)));
```

Comparison:

Method	Type	Characteristics
Poisson	Stochastic	Biologically realistic
Uniform	Deterministic	Stable and repeatable

7 SPICE PWL Generation

7.1 Voltage Mapping

```
Vout = V_low + spike * (V_high - V_low);
```

7.2 Time Expansion

Each timestep ($1\ \mu\text{s}$) is expanded to:

- 10 points of $0.1\ \mu\text{s}$ resolution

```
for k = 0:9
    time_us = base_time + 0.1 * k;
```

Purpose:

- Improve HSPICE convergence
- Generate smooth waveforms

8 Multi-Sample Generation

8.1 Balanced Sampling

```
target = [0 4 7];
numsample = 20;
```

- Select equal number of samples per class
- Avoid dataset bias

8.2 Shuffling Constraint

- No consecutive identical labels

Purpose:

- Prevent temporal bias in training
- Improve learning robustness

9 Output Files

- mnist_8x8_dataset.mat
- mnist_spike_trains.mat
- mnist_digitX.sp

10 Visualization

10.1 Image

```
imshow(images8(:,: ,1), []);
```

10.2 Spike Train

```
imagesc(spikes(:,: ,100));
```

11 Design Insights

- Smaller images reduce hardware complexity
- Poisson encoding enables STDP learning
- Spike timing is critical for synaptic updates
- Dataset ordering affects training behavior

12 Recommended Settings

```
duration_us = 100;  
dt_us = 1;  
max_spikes = 40;  
V_high = 1.2;  
V_low = 0.0;
```

13 Conclusion

This encoding pipeline converts static images into dynamic spike trains suitable for:

- Neuromorphic hardware simulation
- RRAM-based learning systems
- STDP-driven training

It provides a complete interface between **machine learning datasets** and **circuit-level SNN models**.